Previsão de sequências do vírus influenza

Relatório e análise: atualização do código, correção de um erro e comparação entre modelos clássicos e de deep learning

21 de outubro de 2026

Resumo

- O código original da pesquisa foi atualizado para Python 3.12 e versões atuais das bibliotecas (scikit-learn, NumPy, Biopython). O Naive Bayes, que não roda mais nas versões atuais, foi corrigido.
- **Foi encontrado um erro no código original** que tornava todas as sequências-alvo iguais. Com ele, os modelos só precisavam devolver uma sequência fixa, e os resultados ficavam inflados (até 100%). Executando o código com o erro, reproduzi exatamente os valores da Figura 12 da pesquisa original. O erro foi corrigido e todos os experimentos foram refeitos.
- Foram implementados três modelos de deep learning (CNN, Transformer e BiLSTM) e comparados com os quatro modelos clássicos da pesquisa, no mesmo protocolo e com as mesmas métricas. Cada modelo foi executado com 5 sementes aleatórias, e os resultados são apresentados como média ± desvio padrão.
- **Os modelos de deep learning tiveram a maior acurácia por nucleotídeo**: 98,76–99,26% em média, contra 82,22–91,40% dos modelos clássicos.
- **Porém, ficaram muito próximos de simplesmente copiar a sequência da temporada anterior** (98,61%). Nas posições que sofreram mutação, os modelos clássicos acertam uma parcela parecida ou maior, mas à custa de errar muitas posições que não mudaram.
- **Execução única vs. várias sementes:** os modelos de deep learning se mostraram muito estáveis entre sementes (variação máxima de ±0,06 pontos), enquanto as árvores variaram até ±3,00 pontos. Parte das vitórias dos modelos clássicos na execução única desapareceu na média das sementes (Seção 5).
- **Conclusão:** com os dados disponíveis (16 a 35 pares de sequências para treino por conjunto), não é possível afirmar que os modelos de deep learning sejam claramente superiores. O estudo mostra as vantagens e limitações de cada abordagem e indica o que seria necessário para uma comparação conclusiva.

1. Erro no código original

No arquivo `main.py` original, as sequências eram lidas em dois laços:

```
for i in range(len(data_train)):
    train.append(data_train[i].seq)

for j in range(len(data_test)):
    test.append(data_test[i].seq)   # deveria ser data_test[j]
```

O segundo laço percorre `j`, mas lê a sequência com `i`. Em Python, a variável de um laço continua existindo depois que ele termina, com o último valor. Assim, `i` ficava fixo no último índice, e todas as sequências-alvo eram cópias da última sequência do arquivo. O programa rodava sem nenhum aviso.

Consequência para o experimento. Em vez de aprender "dada a sequência de um ano, prever a do ano seguinte", os modelos só precisavam aprender "devolver sempre esta mesma sequência", sem usar a entrada.

Consequência para a métrica. A "acurácia" da pesquisa era o $R^2 \times 100$. Quando todos os alvos são iguais, o scikit-learn atribui $R^2 = 1$ a cada coluna prevista exatamente e $R^2 = 0$ a qualquer diferença, por menor que seja. A nota, portanto, só pode ficar entre 0 e 100%, e as diferenças entre modelos passam a refletir principalmente arredondamentos numéricos, e não a qualidade da previsão.

O código original executado com e sem o erro:

Conjunto	Modelo	Alvos distintos (com erro / corrigido)	Nota com o erro	Nota corrigida
H1N1-HA	Árvore de Decisão	1 / 23	100,00	$-8{,}5 \times 10^{23}$
H1N1-NA	Árvores Extras	1 / 9	100,00	$-1{,}7 \times 10^{25}$
H3N2-HA	Árvores Extras	1 / 37	92,67	$-9{,}4 \times 10^{28}$
H3N2-NA	Floresta Aleatória	1 / 35	91,41	$-4{,}2 \times 10^{26}$

Confirmação. A Figura 12 da pesquisa original (proteína hemaglutinina da cepa H1N1, com dados mais recentes) mostra 100,00% para Árvores de Decisão, 91,23% para Florestas Aleatórias e 100,00% para Árvores Extras. Executando o código original com o erro, obtive **exatamente esses três valores**. Isso confirma que essa figura foi gerada com o erro. O valor se repete com precisão porque, com todos os alvos iguais, a nota depende apenas de arredondamentos numéricos, que não mudam entre execuções.

Para os demais gráficos, não reproduzi os valores um a um. Porém, todos os valores publicados ficam entre 0% e 100%, a faixa possível com o erro; com os alvos corretos, esses modelos dão valores muito negativos. Por isso, é muito provável que os demais gráficos também tenham sido gerados com o erro.

2. Método

2.1 Dados e tarefa

Cada conjunto de dados forma pares entre a sequência *i* de uma temporada mais antiga e a sequência *i* de uma temporada mais nova. O modelo recebe a sequência antiga e deve prever a nova. Foram usados os mesmos quatro conjuntos da pesquisa original:

Conjunto	Entrada	Alvo	Pares	Pares de teste
H1N1-HA	2021	2019–2020	24	8
H1N1-NA	2019–2020	2021	24	8
H3N2-HA	2021	2022	50	15
H3N2-NA	2021	2022	50	15

Nos arquivos de H1N1, algumas sequências incluem bases antes do códon de início do gene (ATG) e outras não, então a posição *i* não corresponde ao mesmo ponto do gene em cada par. Todas as sequências foram alinhadas pelo códon de início antes da análise. O conjunto H1N1-HA, como na pesquisa original, usa 2021 como entrada e 2019–2020 como alvo, ou seja, prevê o passado a partir do futuro.

2.2 Modelos

Modelo	Tipo	Descrição
Cópia da temporada anterior	Referência	Não aprende nada: prevê que a sequência nova é igual à antiga. Serve de régua mínima.
Árvore de Decisão	Clássico	Configuração padrão do scikit-learn.
Floresta Aleatória	Clássico	23 árvores, como na pesquisa original.
Árvores Extras	Clássico	3 árvores, como na pesquisa original.
Naive Bayes	Clássico	Gaussiano; cada bloco de 15 dígitos é uma amostra, como na pesquisa original.
CNN	Deep learning	Rede convolucional 1D com 4 blocos residuais, lendo um nucleotídeo por vez.
Transformer	Deep learning	Codificador Transformer de 4 camadas, lendo a sequência em códons (palavras de 3 letras), técnica de tokenização de PLN.
BiLSTM	Deep learning	LSTM bidirecional de 2 camadas sobre códons, modelo clássico de PLN.

Os modelos clássicos usam a codificação da pesquisa original: A=1, T=2, G=3, C=4, com cada 15 dígitos formando um número. Os modelos de deep learning preveem um nucleotídeo por posição e têm um *mecanismo de cópia* (ideia do CopyNet, em PLN): um peso aprendido que favorece manter o nucleotídeo de entrada, já que cerca de 99% do gene não muda entre temporadas. Configuração de treino: otimizador AdamW, 200 épocas, lotes de 8 pares.

2.3 Avaliação

Todos os modelos usam as mesmas divisões da pesquisa original: 70% dos pares para treino e 30% para teste (`random_state=50`), além de validação cruzada com 2 partes. Cada modelo foi executado com 5 sementes aleatórias (0, 1, 2, 3, 4); os resultados são a média ± desvio padrão entre as sementes. A Cópia e o Naive Bayes não têm componente aleatório, por isso o desvio deles é zero. Métricas:

- **Acurácia por nucleotídeo**: percentual de letras previstas corretamente.
- **Acurácia nas mutações**: acerto apenas nas posições em que a sequência nova difere da antiga. A Cópia sempre tem 0% aqui; a métrica mostra se o modelo prevê mudança.
- As métricas da pesquisa original (R^2, MAE, MSE sobre a codificação de 15 dígitos) também foram calculadas; ver Seção 4.6.

3. O que foi implementado

3.1 Os novos modelos

Os três modelos novos foram escritos em PyTorch e seguem a mesma estrutura: transformam a sequência de entrada em números, processam esses números com uma rede neural e, para cada posição do gene, dão uma pontuação para cada letra (A, T, G, C). A letra com maior pontuação é a previsão.

- **CNN (rede convolucional).** Lê a sequência letra por letra. Filtros deslizam ao longo do DNA e detectam padrões locais. São 4 blocos com filtros de tamanho 9, cada um enxergando mais longe que o anterior (dilatações 1, 2, 4 e 8), de modo que cada posição "vê" cerca de 240 letras de cada lado. Conexões residuais facilitam que a rede aprenda a manter a entrada quando não há mudança.
- **Transformer.** Lê a sequência em códons, isto é, palavras de 3 letras, que é a unidade que codifica um aminoácido. Usa 4 camadas de atenção: cada códon pode "olhar" para todos os outros do gene ao mesmo tempo, o que permite captar mutações que costumam acontecer juntas, mesmo distantes.
- **BiLSTM (LSTM bidirecional).** Modelo clássico de processamento de linguagem natural (PLN). Também lê códons e percorre o gene nos dois sentidos, do início ao fim e do fim ao início, guardando uma memória do que já leu.

Duas técnicas de PLN são compartilhadas pelos três modelos (arquivo `prediction_model/deep/layers.py`):

- **Tokenização por códons** (`codon_tokens`): converte cada trio de letras num único número entre 216 possíveis, como um vocabulário de palavras. Reduz o tamanho da sequência a um terço e trabalha numa unidade com significado biológico.
- **Mecanismo de cópia** (CopyHead): inspirado no CopyNet, usado em PLN para copiar palavras da entrada. Soma à pontuação da letra que já estava na entrada um peso aprendido no treinamento. Como cerca de 99% do gene não muda entre temporadas, "manter a letra" vira o comportamento padrão, e a rede concentra o aprendizado nas posições que mudam.

3.2 Arquivos e funções criados ou alterados

Arquivo	Função ou classe	O que faz
`main.py`	programa principal	Escolhe conjuntos e modelos pela linha de comando, executa a avaliação, imprime os resultados e grava os arquivos CSV.
`dataset.py`	`load_dataset`	Lê os pares de arquivos FASTA e prepara as duas representações: a codificação de 15 dígitos (modelos clássicos) e um número por letra (modelos novos).
	`trim_to_start_codon`	Alinha cada sequência pelo códon de início (ATG), escolhendo o ATG com a maior região codificante.
`evaluation.py`	`evaluate`	Executa um modelo na validação cruzada e na divisão treino/teste, sempre com as mesmas divisões para todos os modelos.
	`score`	Calcula todas as métricas: acurácia por nucleotídeo, acurácia nas mutações, R^2, MAE e MSE.
	`decode_chunks`	Converte as previsões dos modelos clássicos (números de 15 dígitos) de volta em letras, para que todos sejam avaliados com as mesmas métricas.
	`CopyBaseline`	O modelo de referência "Cópia da temporada anterior".
`prediction_model/deep/`	`ResidualCNN`	A CNN.
	`CodonTransformer`	O Transformer.
	`CodonBiLSTM`	O BiLSTM.
	`NeuralSequenceModel`	Treina e usa qualquer uma das redes, com a mesma interface dos modelos do scikit-learn, para que todos passem pela mesma avaliação.
`prediction_model/naiveBayes.py`	`ChunkNaiveBayes`	Naive Bayes corrigido para as versões atuais do scikit-learn (converte os rótulos para números inteiros).
`encoding_data.py`	`EncodingData`	Codificação original (A=1, T=2, G=3, C=4, blocos de 15 dígitos), reorganizada para ser reutilizada na avaliação.

Principais opções de linha de comando:

Opção	O que faz
`--dataset`	Escolhe o conjunto (h1n1-ha, h1n1-na, h3n2-ha, h3n2-na ou all).
`--models`	Escolhe os modelos (por exemplo, classical, deep ou nomes separados por vírgula).
`--align`	Alinha as sequências pelo códon de início.
`--seeds`	Repete cada modelo com várias sementes e grava média e desvio padrão.
`--epochs`	Número de épocas de treino dos modelos de deep learning.

4. Resultados

Valores em %, média ± desvio padrão entre 5 sementes, no conjunto de teste. Em negrito, a maior média de cada coluna.

4.1 Acurácia por nucleotídeo

Modelo	Tipo	H1N1-HA	H1N1-NA	H3N2-HA	H3N2-NA	Média
Cópia da temporada anterior	Referência	97,44 ± 0,00	98,18 ± 0,00	99,26 ± 0,00	99,55 ± 0,00	98,61 ± 0,00
Árvore de Decisão	Clássico	94,54 ± 0,00	97,73 ± 1,41	83,40 ± 0,00	89,92 ± 0,00	91,40 ± 0,35
Floresta Aleatória	Clássico	86,49 ± 0,38	80,43 ± 3,00	80,66 ± 0,56	88,60 ± 0,35	84,05 ± 0,77
Árvores Extras	Clássico	93,72 ± 0,72	92,01 ± 2,67	83,40 ± 0,00	89,92 ± 0,00	89,76 ± 0,67
Naive Bayes	Clássico	74,27 ± 0,00	94,04 ± 0,00	63,84 ± 0,00	96,73 ± 0,00	82,22 ± 0,00
CNN	Deep learning	99,23 ± 0,01	96,89 ± 0,06	**99,30 ± 0,00**	**99,61 ± 0,00**	98,76 ± 0,02
Transformer	Deep learning	99,29 ± 0,01	**98,83 ± 0,04**	99,30 ± 0,00	99,61 ± 0,00	**99,26 ± 0,01**
BiLSTM	Deep learning	**99,31 ± 0,01**	98,55 ± 0,05	**99,30 ± 0,00**	**99,61 ± 0,00**	99,19 ± 0,01

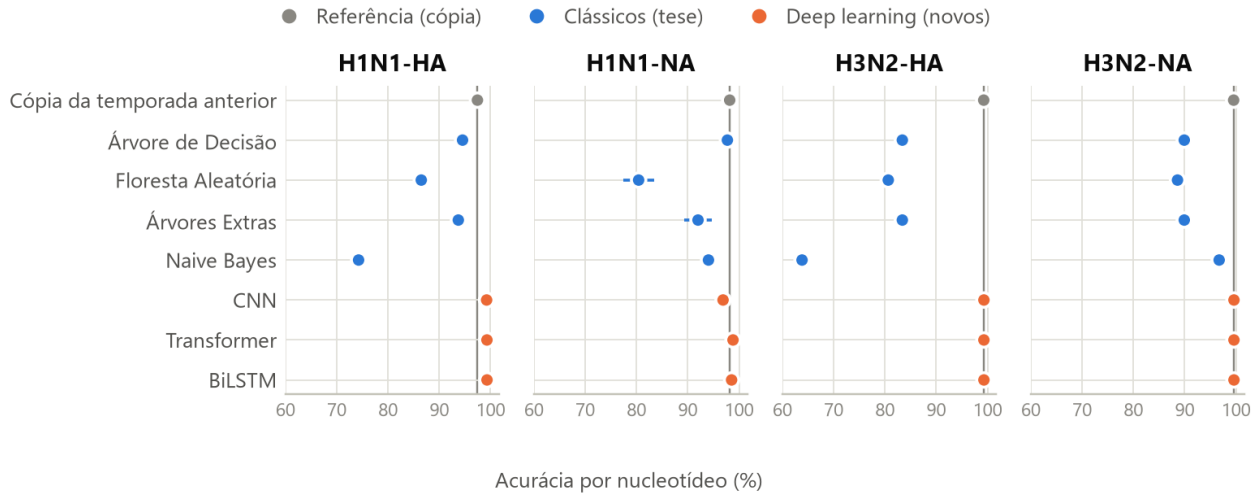


Figura 1. Acurácia por nucleotídeo (média e desvio padrão entre sementes). A linha cinza vertical marca a Cópia da temporada anterior.

4.2 Acurácia nas mutações

Modelo	Tipo	H1N1-HA	H1N1-NA	H3N2-HA	H3N2-NA	Média
Cópia da temporada anterior	Referência	0,00 ± 0,00	0,00 ± 0,00	0,00 ± 0,00	0,00 ± 0,00	0,00 ± 0,00
Árvore de Decisão	Clássico	66,40 ± 0,00	61,18 ± 3,17	24,47 ± 0,00	**15,62 ± 0,00**	**41,92 ± 0,79**
Floresta Aleatória	Clássico	54,74 ± 1,97	44,51 ± 2,56	20,11 ± 3,45	12,08 ± 0,83	32,86 ± 0,94
Árvores Extras	Clássico	60,81 ± 1,04	49,02 ± 2,26	24,47 ± 0,00	**15,62 ± 0,00**	37,48 ± 0,43
Naive Bayes	Clássico	54,25 ± 0,00	**63,73 ± 0,00**	**28,72 ± 0,00**	12,50 ± 0,00	39,80 ± 0,00
CNN	Deep learning	**68,83 ± 0,72**	62,94 ± 0,85	7,98 ± 0,00	13,75 ± 0,42	38,37 ± 0,35
Transformer	Deep learning	63,97 ± 0,00	52,75 ± 0,66	7,45 ± 0,00	12,29 ± 1,02	34,11 ± 0,34
BiLSTM	Deep learning	64,78 ± 0,00	52,84 ± 0,72	7,98 ± 0,00	12,71 ± 0,42	34,58 ± 0,14

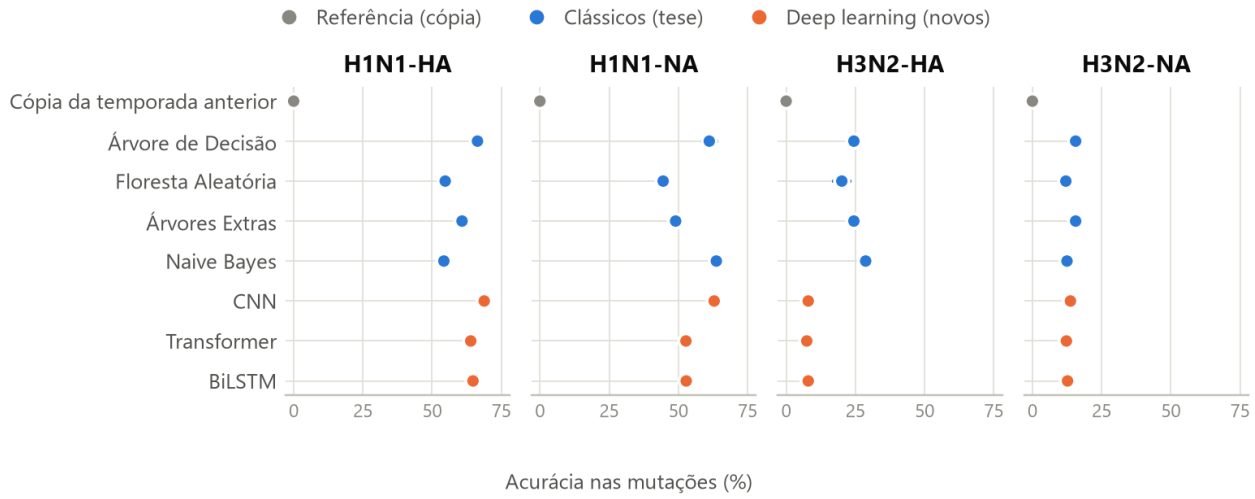


Figura 2. Acurácia nas posições que sofreram mutação (média e desvio padrão entre sementes). A Cópia sempre tem 0%.

4.3 O equilíbrio entre as duas métricas

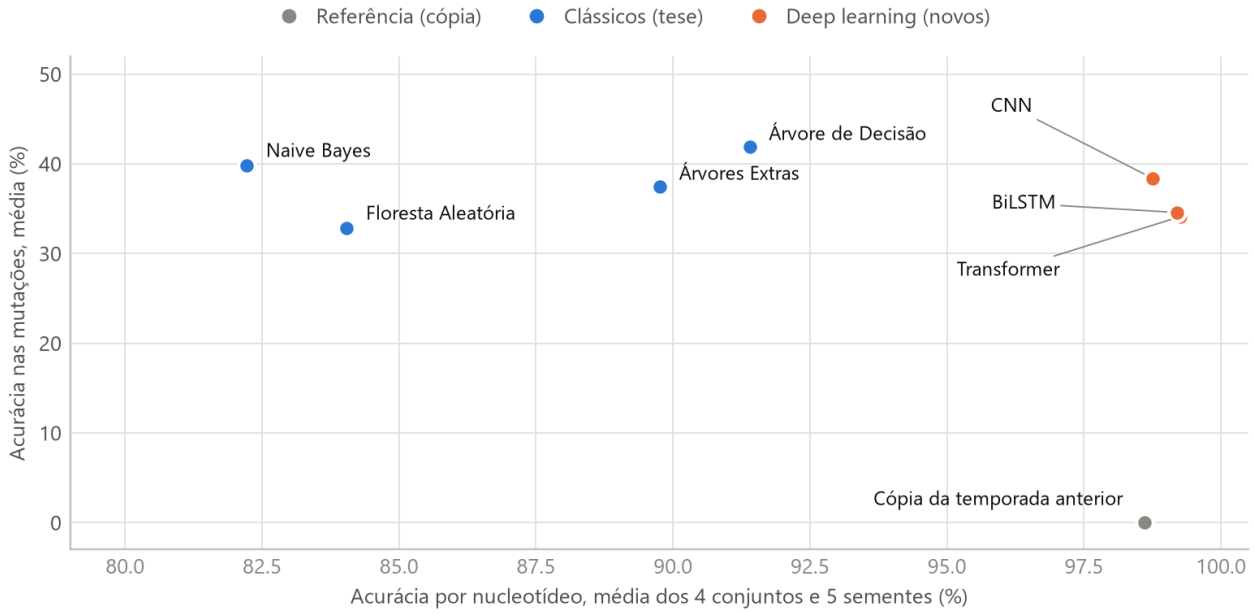


Figura 3. Acurácia geral contra acurácia nas mutações, na média dos quatro conjuntos. Quanto mais para cima e para a direita, melhor.

4.4 Os modelos de deep learning superam a Cópia?

Número de sementes em que cada modelo de deep learning teve acurácia por nucleotídeo maior que a Cópia da temporada anterior. Isso indica se a pequena vantagem é consistente ou depende da sorte do treinamento.

Modelo	H1N1-HA	H1N1-NA	H3N2-HA	H3N2-NA	Média
CNN	5 de 5	0 de 5	5 de 5	5 de 5	5 de 5
Transformer	5 de 5	5 de 5	5 de 5	5 de 5	5 de 5
BiLSTM	5 de 5	5 de 5	5 de 5	5 de 5	5 de 5

4.5 Validação cruzada

Acurácia por nucleotídeo na validação cruzada (cada parte treina com metade dos pares, ou seja, com ainda menos dados).

Modelo	Tipo	H1N1-HA	H1N1-NA	H3N2-HA	H3N2-NA	Média
Cópia da temporada anterior	Referência	95,23 ± 0,00	97,44 ± 0,00	99,35 ± 0,00	**99,62 ± 0,00**	97,91 ± 0,00
Árvore de Decisão	Clássico	96,31 ± 1,52	95,93 ± 2,18	86,35 ± 0,28	85,28 ± 0,00	90,97 ± 0,67
Floresta Aleatória	Clássico	84,91 ± 0,74	76,62 ± 9,20	82,26 ± 0,16	84,18 ± 0,19	81,99 ± 2,28
Árvores Extras	Clássico	93,26 ± 0,50	86,02 ± 4,82	85,71 ± 0,49	85,28 ± 0,00	87,57 ± 1,23
Naive Bayes	Clássico	74,57 ± 0,00	93,23 ± 0,00	62,42 ± 0,00	97,22 ± 0,00	81,86 ± 0,00
CNN	Deep learning	96,78 ± 0,02	97,14 ± 0,01	**99,35 ± 0,01**	99,56 ± 0,02	98,21 ± 0,01
Transformer	Deep learning	**96,96 ± 0,02**	**97,84 ± 0,08**	99,33 ± 0,01	99,55 ± 0,01	**98,42 ± 0,02**
BiLSTM	Deep learning	96,86 ± 0,01	97,70 ± 0,04	99,34 ± 0,01	99,56 ± 0,02	98,37 ± 0,01

4.6 A métrica original (R^2)

O R^2 sobre a codificação de 15 dígitos não diferencia os modelos: o R^2 ponderado pela variância fica igual ou abaixo de zero para todos, e o R^2 simples chega a valores como -10^{25}. A causa é a própria codificação: um erro no primeiro dígito de um bloco altera o número em trilhões, e um erro no último, em uma unidade. O tamanho do erro depende da posição, não da quantidade de erros. Por isso, as acurácias por nucleotídeo e nas mutações são medidas mais adequadas para esta tarefa.

5. Execução única vs. várias sementes

Os modelos com componente aleatório (as árvores e os três modelos de deep learning) dependem da semente aleatória: ela define, por exemplo, quais amostras cada árvore sorteia e os pesos iniciais de cada rede neural. A primeira análise rodou cada modelo **uma única vez** (semente 0). Em seguida, cada modelo foi executado com **5 sementes**, para separar o desempenho real do efeito da sorte. A tabela compara as duas análises na média dos quatro conjuntos.

Modelo	Acurácia geral: execução única	Acurácia geral: 5 sementes	Mutações: execução única	Mutações: 5 sementes
Cópia da temporada anterior	98,61	98,61 ± 0,00	0,00	0,00 ± 0,00
Árvore de Decisão	91,67	91,40 ± 0,35	42,43	41,92 ± 0,79
Floresta Aleatória	83,51	84,05 ± 0,77	32,56	32,86 ± 0,94
Árvores Extras	90,10	89,76 ± 0,67	37,58	37,48 ± 0,43
Naive Bayes	82,22	82,22 ± 0,00	39,80	39,80 ± 0,00
CNN	98,76	98,76 ± 0,02	38,15	38,37 ± 0,35
Transformer	99,25	99,26 ± 0,01	33,83	34,11 ± 0,34
BiLSTM	99,18	99,19 ± 0,01	34,43	34,58 ± 0,14

Estabilidade. Os modelos de deep learning variaram no máximo ±0,06 pontos de acurácia geral entre sementes, em qualquer conjunto. As árvores variaram até ±3,00 pontos. Ou seja, os resultados dos modelos novos praticamente não dependem da sorte do treinamento; os das árvores dependem bastante.

Vencedores que eram sorte. Na acurácia geral, em H1N1-NA, o melhor modelo na execução única era Árvore de Decisão (98,82%), mas na média das sementes é Transformer (98,83%, contra 97,73% de Árvore de

Decisão).

Validação cruzada. Na validação cruzada, o tipo de modelo vencedor mudou em: H1N1-HA (execução única: Árvore de Decisão; média das sementes: Transformer); H1N1-NA (execução única: Árvore de Decisão; média das sementes: Transformer). A conclusão da execução única, nesses conjuntos, dependia da semente.

Por que isso importa. Com uma única execução, uma diferença pequena entre dois modelos pode ser apenas efeito da semente. A média de várias sementes, com o desvio padrão, mostra quais diferenças se repetem e quais não. Por isso, as conclusões deste relatório se baseiam nas médias.

6. Análise e conclusão

Acurácia geral. O melhor modelo de deep learning (Transformer, 99,26%) supera o melhor modelo clássico (Árvore de Decisão, 91,40%) por 7,9 pontos na média dos quatro conjuntos. Mas fica apenas 0,65 pontos acima da Cópia da temporada anterior (98,61%): na prática, os modelos de deep learning aprendem a copiar a entrada e alteram poucas posições.

Mutações. Na média, o modelo que mais acerta mutações é Árvore de Decisão (41,9%); entre os de deep learning, CNN (38,4%). Os modelos clássicos não aprendem a transformar a sequência de entrada: aprendem a devolver algo parecido com uma sequência típica da temporada nova. Isso acerta várias mutações, mas também altera posições que já estavam corretas, o que derruba a acurácia geral.

Por que os modelos de deep learning não se destacam mais. (1) Há muito poucos dados: cada modelo treina com 16 a 35 pares de sequências. (2) Os pares são formados pela ordem no arquivo, e não por ancestralidade; parte das "mutações" de um par são diferenças entre cepas não relacionadas, impossíveis de prever a partir da entrada.

Conclusão. Com os dados disponíveis, os modelos de deep learning foram os mais precisos na média, mas a vantagem sobre simplesmente repetir a temporada anterior é pequena, e os modelos clássicos acertam uma parcela parecida ou maior das mutações. Não é possível afirmar que uma abordagem seja claramente superior. O resultado mostra que, neste volume de dados, a escolha do modelo importa menos do que a quantidade e a organização dos dados.

7. Limitações e próximos passos

- Os conjuntos de teste são pequenos (8 pares em H1N1, 15 em H3N2).
- Não foram feitos testes de significância estatística; a consistência entre sementes (Seção 4.4) foi usada como indicação.
- Os hiperparâmetros dos modelos de deep learning não foram ajustados.
- A pesquisa original também coletou arquivos maiores, com milhares de sequências de H3N2 de 2021 e 2022. Depois de remover repetidas e parciais, restam cerca de 220 sequências de 2021 e 1.100 de 2022 por gene. Pareando cada sequência nova com a antiga mais parecida, seria possível treinar com cerca de 1.100 pares, o que permitiria uma comparação mais conclusiva.

Apêndice: como reproduzir

Na pasta do projeto, com o ambiente descrito no README.md:

```
python main.py --dataset all --align --seeds 5
```

Os resultados são gravados em `results/` (arquivos `*-aligned-seeds.csv` e `*-aligned-seeds-summary.csv`). Todas as tabelas e gráficos deste relatório são gerados a partir desses arquivos. Ambiente: Python 3.12.10, scikit-learn 1.9.1, NumPy 2.5.4, Biopython 1.88, PyTorch 2.14.1, GPU NVIDIA GeForce MX570 A.